EXAMEN DE SESSION NORMALE (SN) - SUPÉRIEUR

Session Normale (SN) - INF 101 : Algorithmique et Langage C (Niveau 1 Ingénieur) - Année 2023-2024

Épreuve : Algorithmique et Programmation C | Niveau : Niveau 1 Ingénieur | Durée : 3h00 | Barème : 20 pts

CONSIGNES GÉNÉRALES : La clarté, la rigueur de la démarche et la précision du vocabulaire académique sont impératives.

EXERCICE 1 : ARITHMÉTIQUE DES POINTEURS ET MODÈLE MÉMOIRE (5 points)

Compréhension fine de l'organisation mémoire (Stack, Heap, Data, BSS) et de l'arithmétique des pointeurs sous architecture 64 bits.

```
int tab[5] = {10, 25, 42, 77, 99};
int *ptr = tab + 2;
printf("%d, %d, %ld\n", *ptr, *(ptr - 1), ptr - tab);
```

1. Analyse du fragment de code ci-dessus :

a) Qu'affiche exactement le fragment de code C lors de son exécution sous Linux 64 bits ? Justifier chaque valeur.

b) Expliquer la différence entre static int count = 0 déclaré à l'intérieur d'une fonction et int count = 0 (automatique).

c) Pourquoi l'expression sizeof(ptr) retourne-t-elle 8 alors que sizeof(tab) retourne 20 ?

2. Débogage de fuite mémoire (Valgrind) :

a) Identifier l'erreur fatale dans : char *s = malloc(10); strcpy(s, "Campus360"); free(s); printf("%s", s);

b) Expliquer ce qu'est un déréférencement de pointeur pendant (dangling pointer) et comment l'éviter systématiquement.

EXERCICE 2 : STRUCTURES DE DONNÉES DYNAMIQUES - LISTE DOUBLEMENT CHAÎNÉE (7 points)

Conception du moteur d'ordonnancement de tâches temps réel pour un automate industriel développé au laboratoire de génie informatique de l'ENSPY.

```
typedef struct Task {
    int id;
    char name[32];
    int priority;
    struct Task *prev;
    struct Task *next;
} Task;

typedef struct Scheduler {
    Task *head;
    Task *tail;
    int count;
} Scheduler;
```

1. Fonctions de gestion de la liste chaînée :

a) Écrire la fonction Task* create_task(int id, const char *name, int prio) avec allocation dynamique et vérification de NULL.

b) Écrire la fonction void insert_sorted(Scheduler *s, Task *t) insérant la tâche par ordre décroissant de priorité.

c) Écrire la fonction Task* pop_highest_priority(Scheduler *s) qui extrait la tâche en tête de liste et met à jour les pointeurs.

d) Écrire la fonction void free_scheduler(Scheduler *s) assurant la libération intégrale sans mémoire orpheline.

EXERCICE 3 : ARBRES BINAIRES DE RECHERCHE (ABR) & DICTIONNAIRE (8 points)

Indexation lexicographique des matricules d'étudiants de l'ENSPY pour une recherche en temps O(log n).

1. Définition et insertion dans l'ABR :

a) Définir la structure ABRNode contenant le matricule (chaîne de 12 caractères), la moyenne (float) et les fils gauche/droit.

b) Écrire en langage C la fonction récursive ABRNode* insert_node(ABRNode *root, const char *mat, float moy).

c) Écrire la fonction ABRNode* search_node(ABRNode *root, const char *mat) retournant le pointeur vers l'étudiant ou NULL.

2. Parcours et propriétés de l'arbre :

a) Écrire la fonction void print_inorder(ABRNode *root) affichant tous les matricules par ordre alphabétique croissant.

b) Écrire la fonction int tree_height(ABRNode *root) calculant récursivement la hauteur maximale de l'arbre.

c) Quel est le pire cas de complexité d'un ABR et comment un arbre AVL permet-il de garantir une complexité logarithmique ?